

Your Course Code - Course Title

SmartCross

Real-Time Railway Crossing Safety and Prediction System

102XXXXXXX Student Name One
102XXXXXXX Student Name Two
102XXXXXXX Student Name Three

Group: XXXX BE Third Year, CSE

Faculty Advisor
Dr. Faculty Member Name

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.2	Problem Statement	2
1.3	Project Objectives	2
2	Methodology and Design	4
2.1	System Architecture	4
2.1.1	High-Level Design	4
2.1.2	Technical Stack	4
2.2	Implementation Details	5
2.2.1	Live Train Data Module	5
2.2.2	Train-Crossing Prediction Module	5
2.2.3	Crossing Status Module	5
2.2.4	User Notification Module	5
3	Requirements and UML Design	6
3.1	Functional Requirements	6
3.2	Non-Functional Requirements	6
3.3	Use Case Diagram	6
3.4	UML Component Diagram	7
3.5	Activity Diagram	7
3.6	Class Diagram	9
3.7	SDLC Diagram	9
3.7.1	SDLC Stages	10
4	Results and Evaluation	11
4.1	Testing Methodology	11
4.2	Performance Metrics	11
4.3	Analysis	11
5	Conclusion	13
5.1	Summary of Work	13
5.2	Key Findings	13
5.3	Future Work and Recommendations	13
5.4	Final Remarks	14

Chapter 1

Introduction

1.1 Background and Context

Railway level crossings are locations where road users and railway traffic share the same point of intersection. At an unmanned or busy crossing, a user may not know when a train is approaching, how long the crossing is expected to remain closed, or whether it is safe to proceed.

SmartCross is proposed as a real-time railway crossing information and safety system. It combines live train information with railway-crossing information to estimate the arrival of a train at a crossing and provide useful status information to the user.

The objective is not merely to display the location of a train. SmartCross processes live train information and converts it into a user-oriented prediction such as whether a crossing is open, likely to close soon, currently closed, or likely to open soon.

1.2 Problem Statement

A road user approaching a railway crossing generally has limited information about the current or expected state of the crossing. Existing train-tracking systems may provide train location but do not necessarily answer the more practical question: “What is going to happen at the crossing I am approaching?”

SmartCross addresses this problem by connecting live train information with crossing-specific logic and presenting the resulting prediction through a web application.

1.3 Project Objectives

1. Develop a system that receives live train information from an external railway data source.
2. Map relevant train movement to railway crossings and estimate the expected arrival time of a train at the crossing.
3. Predict and display the crossing state, including open, closing soon, closed, and opening soon.
4. Provide estimated waiting information so that users can make safer travel decisions.

5. Build a scalable backend using Node.js, Express, and MongoDB with a web-based user interface.

Chapter 2

Methodology and Design

2.1 System Architecture

SmartCross follows a client-server architecture. The backend receives live train data, processes the train's position and movement, calculates an estimated time to the relevant railway crossing, and generates a crossing-status prediction. The frontend presents this information to the user.

2.1.1 High-Level Design

Figure 2.1 shows the major components of the SmartCross system.

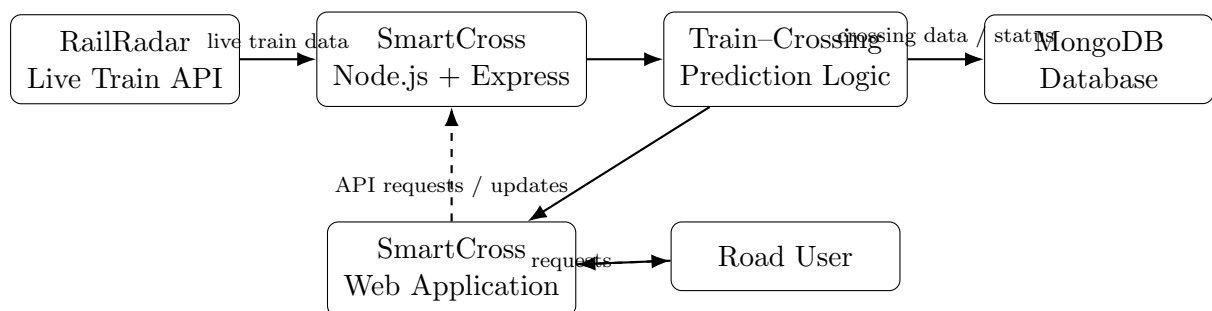


Figure 2.1: High-Level SmartCross System Architecture

2.1.2 Technical Stack

- **Backend:** Node.js and Express.js.
- **Database:** MongoDB.
- **External Data Source:** RailRadar live train API.
- **Frontend:** Web-based React application.
- **Real-Time Communication:** WebSocket/Socket.IO can be used for pushing updated status to connected clients.
- **Development and Testing:** REST API testing tools and local development environment.

2.2 Implementation Details

2.2.1 Live Train Data Module

The live train data module obtains information about the relevant train from the external railway API. The backend validates and processes the received information before using it for crossing prediction.

A mock-data mode can also be maintained during development so that the application can be tested when live API access is unavailable.

2.2.2 Train–Crossing Prediction Module

The prediction module determines the expected time at which a train will affect a particular crossing. A simplified estimation can be represented as:

$$T_{\text{arrival}} = \frac{D_{\text{crossing}}}{V_{\text{train}}}$$

where D_{crossing} is the relevant distance between the train and the crossing and V_{train} is the estimated train speed.

The system can then compare the estimated arrival time with configured crossing timings to determine an appropriate user-facing state.

2.2.3 Crossing Status Module

The crossing-status module converts the prediction into understandable states:

- **Open:** No immediate train-related closure is expected.
- **Closing Soon:** A relevant train is expected to reach the crossing soon.
- **Closed:** The crossing is expected to be unavailable because of the train.
- **Opening Soon:** The train has passed or is sufficiently clear and the crossing is expected to become available.

2.2.4 User Notification Module

The frontend presents the crossing state, expected train information, and estimated waiting time. This allows a road user to decide whether to wait or consider an alternative route.

Chapter 3

Requirements and UML Design

3.1 Functional Requirements

1. The system shall obtain live train information.
2. The system shall identify the relevant railway crossing for a train.
3. The system shall estimate train arrival time at the crossing.
4. The system shall determine a predicted crossing status.
5. The system shall display crossing status to the user.
6. The system shall display relevant train and waiting-time information.
7. The system shall store required crossing and system data in MongoDB.

3.2 Non-Functional Requirements

- **Availability:** The system should continue to provide useful information whenever valid train data is available.
- **Performance:** API responses and status updates should be generated quickly enough for real-time use.
- **Scalability:** The backend should support multiple crossings and multiple users.
- **Usability:** Crossing status should be easy to understand at a glance.
- **Reliability:** Invalid or unavailable external data should be handled gracefully.

3.3 Use Case Diagram

The use case diagram identifies the major interactions between SmartCross users and the system.

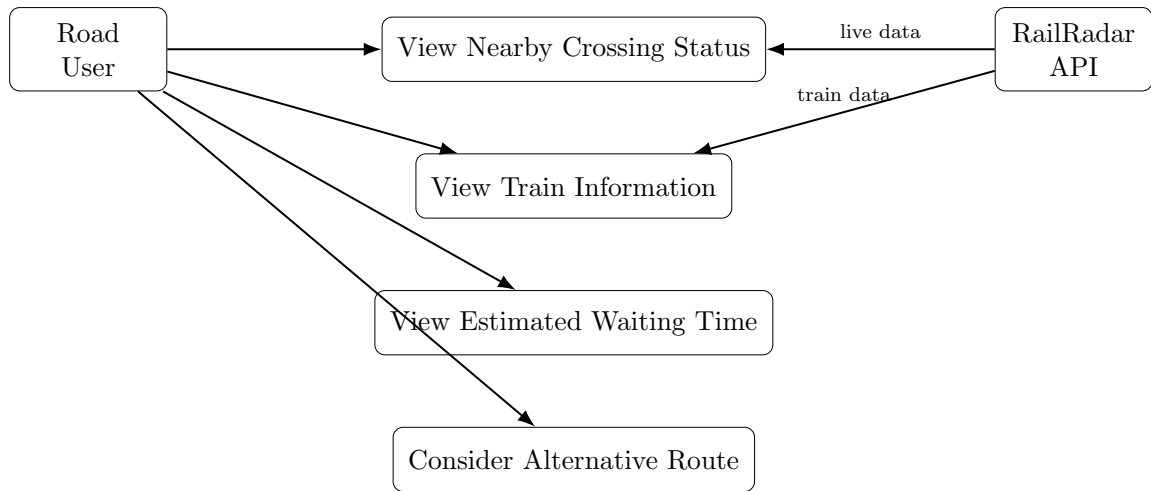


Figure 3.1: SmartCross Use Case Diagram

3.4 UML Component Diagram

The following UML-oriented component view shows how the principal software components communicate.

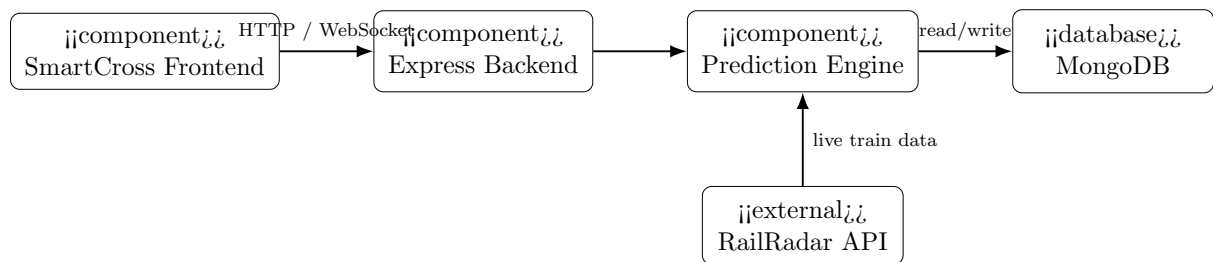


Figure 3.2: UML Component View of SmartCross

3.5 Activity Diagram

The activity diagram describes the flow from receiving live train data to showing the crossing status to a road user.

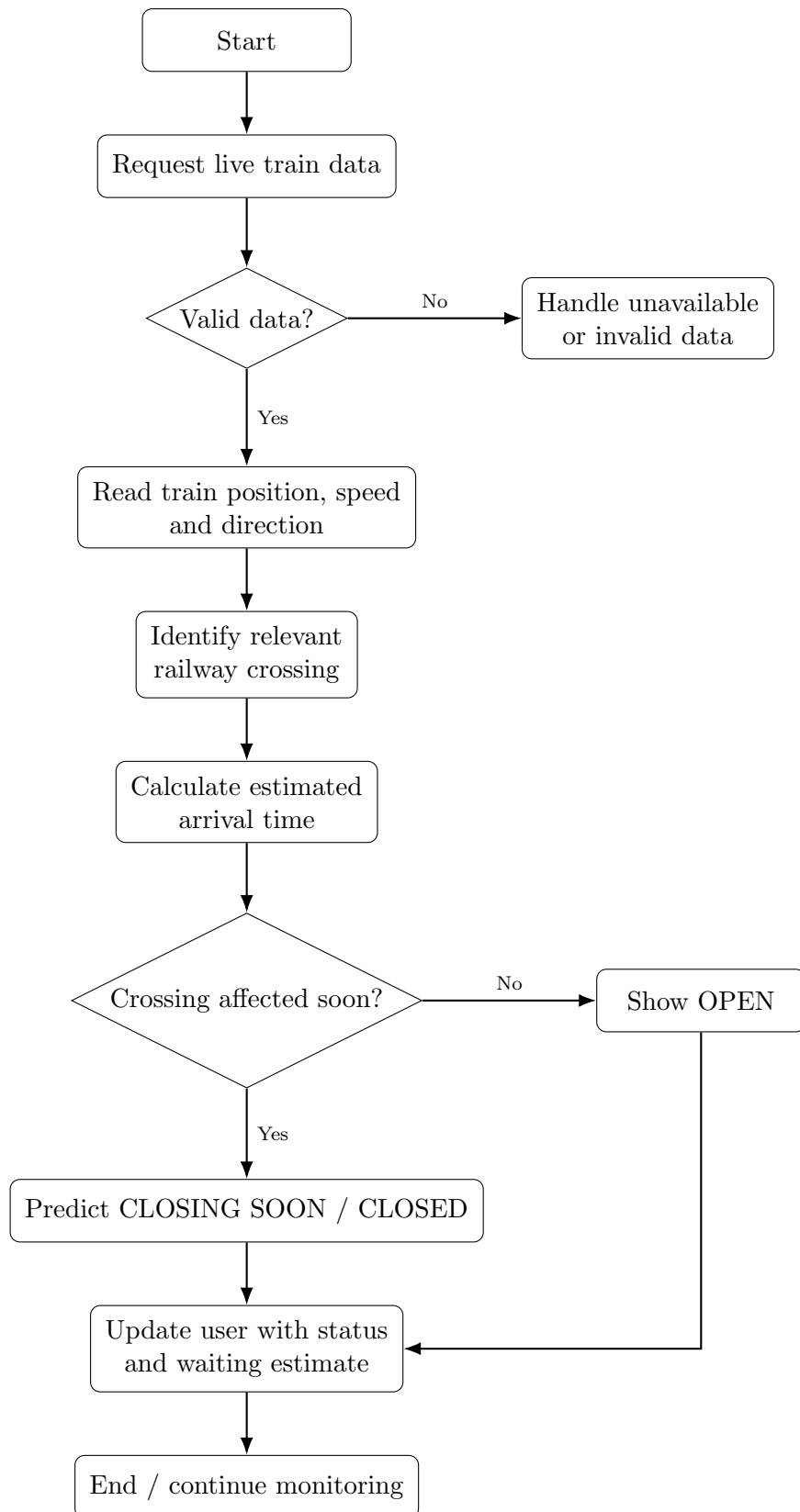


Figure 3.3: SmartCross Activity Diagram

3.6 Class Diagram

The class diagram presents the principal entities used by the SmartCross application. The exact implementation classes may be refined during development.

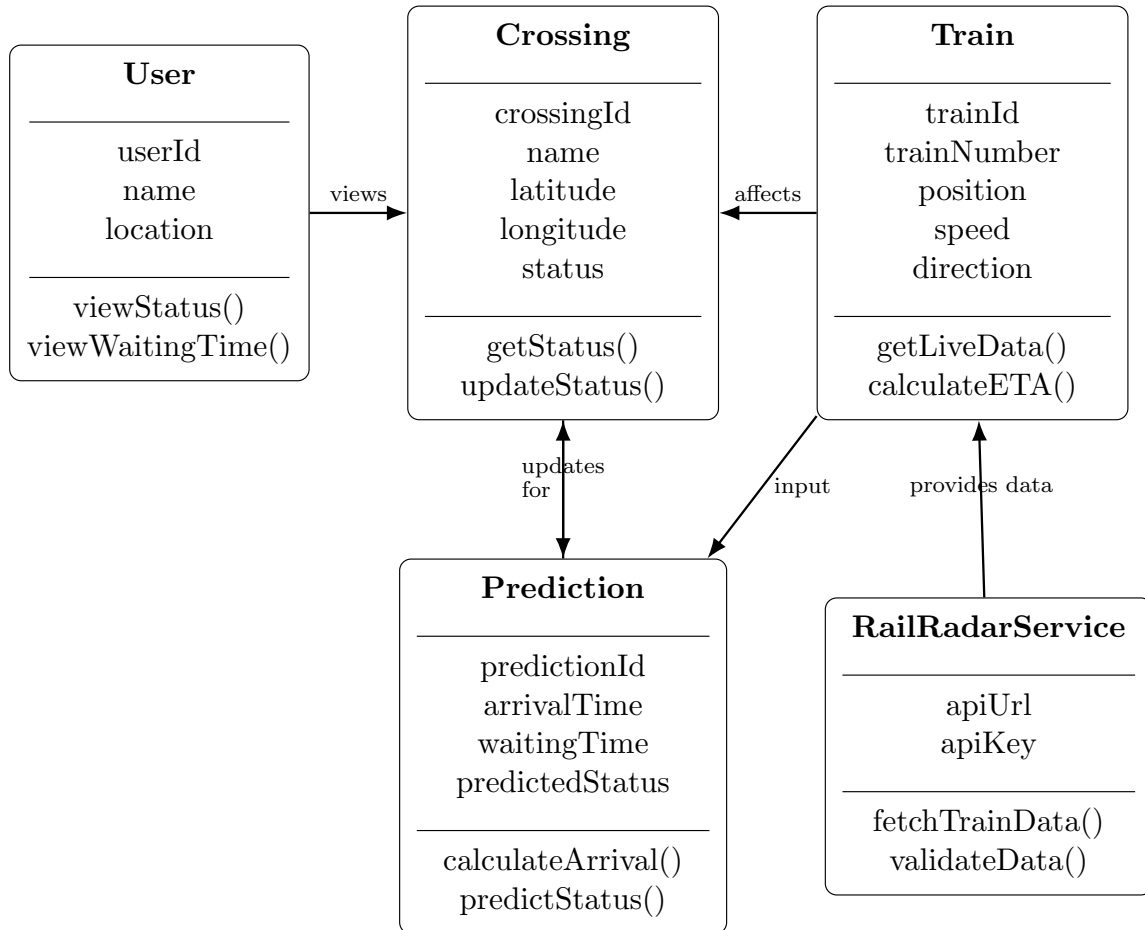


Figure 3.4: SmartCross Class Diagram

3.7 SDLC Diagram

SmartCross follows an iterative Software Development Life Cycle (SDLC). Each stage produces inputs for the next stage and feedback can be used to improve earlier stages.

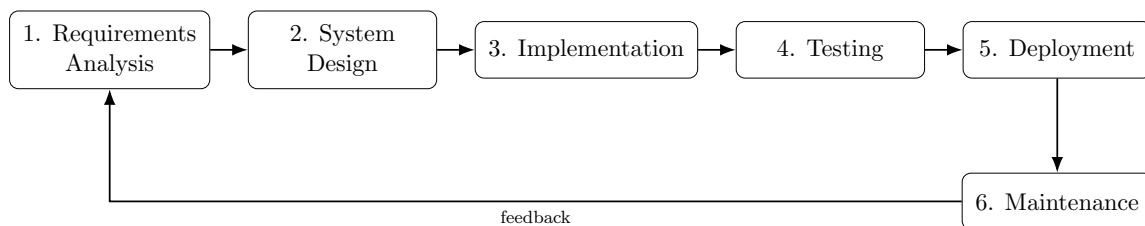


Figure 3.5: SDLC Model Used for SmartCross

3.7.1 SDLC Stages

1. **Requirements Analysis:** Identify the railway-crossing safety problem and define system requirements.
2. **System Design:** Design the frontend, backend, database, prediction logic, APIs, and UML models.
3. **Implementation:** Develop the Node.js/Express backend, MongoDB models, frontend, API integration, and prediction logic.
4. **Testing:** Perform unit, integration, API, and user-interface testing.
5. **Deployment:** Deploy the web application and configure the production environment.
6. **Maintenance:** Monitor the system, handle API changes, fix bugs, and improve prediction and user experience.

Chapter 4

Results and Evaluation

4.1 Testing Methodology

The SmartCross system can be evaluated using the following testing levels:

- **Unit Testing:** Test individual prediction, validation, and database functions.
- **Integration Testing:** Verify that the frontend, backend, database, and external train API work together.
- **API Testing:** Test live and mock train-data responses and verify handling of invalid or unavailable responses.
- **Performance Testing:** Measure response time and behaviour under multiple requests.
- **User Testing:** Verify that crossing status and waiting information are understandable to users.

4.2 Performance Metrics

Actual measured values should be inserted after testing. The following table provides a structure for recording them.

Metric	Target	Achieved
API Response Time	To be measured	To be measured
Prediction Response Time	To be measured	To be measured
Status Accuracy	To be measured	To be measured
System Availability	To be measured	To be measured

Table 4.1: SmartCross Performance Evaluation

4.3 Analysis

The main evaluation criteria are whether SmartCross can correctly process valid train information, identify the relevant crossing, estimate arrival time, and present a useful crossing status to the user.

Special attention should be given to API failures, stale train positions, incorrect speed information, and situations in which live train data is not available.

Chapter 5

Conclusion

5.1 Summary of Work

SmartCross is designed as a railway-crossing safety and information platform that connects live train information with crossing-specific prediction logic. Instead of requiring a user to interpret raw train-tracking data, the system aims to present an understandable crossing state and estimated waiting information.

The project architecture uses a Node.js and Express backend, MongoDB for data storage, a web-based frontend, and an external live train data source.

5.2 Key Findings

- Live train data can be converted into crossing-specific information.
- ETA-based prediction provides more useful information to a road user than raw train location alone.
- A modular backend makes it possible to add more railway crossings and additional data sources later.
- Proper handling of unavailable or invalid external data is important for a real-time safety-oriented application.

5.3 Future Work and Recommendations

1. Improve ETA prediction by incorporating historical train movement and additional railway data.
2. Add GPS-based detection of the user's approach to a crossing.
3. Add route suggestions when a crossing is expected to remain closed.
4. Support multiple railway data providers for better availability.
5. Add push notifications for closing-soon and opening-soon events.
6. Improve scalability for a larger number of crossings and concurrent users.
7. Evaluate prediction accuracy using real-world historical train and crossing data.

5.4 Final Remarks

SmartCross focuses on converting real-time railway information into practical, easy-to-understand information for people approaching railway crossings. With further testing and real-world data validation, the system can be extended into a more comprehensive railway-crossing assistance platform.